La FAQ dans les poids ou dans le contexte : ce que l'on gagne à l'usage

Expérience 01 (FAQ « Mon espace santé ») : temps de réponse, capacité et tokens sur un DGX Spark, 13 novembre 2026 (version 2 : jusqu'à 1 024 requêtes simultanées, plafonds atteints)

---

1. La question

Le modèle entraîné de l'expérience 01 (Gemma 4 31B, version 5 bits pour RTX 5090) connaît la FAQ « Mon espace santé » sans l'avoir sous les yeux. La solution sans entraînement consiste à donner au même modèle, avant entraînement, la FAQ entière dans son contexte à chaque question. Les deux répondent presque aussi bien (expérience 01, 704 questions de test jamais vues) :

Modèle	Strict	Complétude	Fidélité
Gemma 4 31B entraîné, 5 bits (modèle RTX 5090)	81,1 %	75,7 %	97,4 %
Gemma 4 31B de départ, FAQ entière dans le contexte	79,8 %	79,0 %	99,6 %

Ce document chiffre ce que coûte chacune des deux solutions à l'usage, sur un DGX Spark (puce GB10, 128 Go de mémoire unifiée à 273 Go/s) qui sert les usagers avec vLLM. Chaque usager pose **une seule question** (pas de conversation). On fait varier le nombre de requêtes traitées en même temps, de 1 à 1 024, jusqu'à atteindre le plafond de la machine.

Trois façons de servir sont comparées :

- **Modèle entraîné** : il reçoit une consigne courte (« Tu es l'assistant de Mon espace santé. ») et la question.
- **Modèle de départ + FAQ, sans cache** : chaque requête contient la FAQ entière, recalculée à chaque fois.
- **Modèle de départ + FAQ, cache de préfixe** : la FAQ, identique pour tous, est calculée une fois et gardée en mémoire par vLLM (option activée par défaut). C'est la meilleure configuration possible sans entraînement.

---

2. Résumé

	Modèle entraîné	FAQ, cache de préfixe	FAQ, sans cache
Tokens à lire par question	46	15 341	15 341
Tokens traités par échange, réponse comprise	192	15 474	15 474
Réponse complète, un usager seul	22 s	21 s	31 s
Réponse complète, 64 requêtes en cours	30 s	90 s	663 s
Plafond : réponses servies par heure	**environ 15 700**	**environ 3 000**	**environ 350**
Plafond atteint vers	256 requêtes en cours	256 requêtes en cours	32 requêtes en cours

- **Tokens** : le modèle entraîné lit 333 fois moins de tokens par question et traite au total 81 fois moins de tokens par échange.
- **Un usager seul** : avec le cache de préfixe, aucune différence de temps. Sans cache, la lecture de la FAQ ajoute 10 secondes avant le premier mot.
- **Plafond** : un DGX Spark sert au plus environ 15 700 réponses par heure avec le modèle entraîné, 5,2 fois plus qu'avec la FAQ en cache (environ 3 000) et 45 fois plus que sans cache (environ 350). Pour servir autant de questions, il faudrait environ 5 Spark avec la FAQ en cache, ou 45 sans cache, pour un seul avec le modèle entraîné.
- **Pourquoi** : sans cache, chaque question oblige à recalculer la FAQ (environ 10 secondes de calcul sur le Spark), d'où un plafond de 3 600 / 10 ≈ 350 réponses par heure. Avec le cache, le calcul disparaît mais pas la lecture mémoire : à chaque mot généré, le Spark relit le cache de la FAQ de chaque usager (environ 1,5 Go chacun), alors que le modèle entraîné n'a que 0,17 Go par usager à relire.
- Les rapports entre les trois solutions changent peu selon les hypothèses : 5,0 à 5,5 fois entre le modèle entraîné et la FAQ en cache, 38 à 67 fois avec la FAQ sans cache.

Les temps Spark sont des **estimations** : nous n'avons pas de DGX Spark. Ils viennent d'un simulateur de serveur vLLM validé sur 28 mesures réelles faites sur des H100 avec le même modèle et le même logiciel, dont 4 mesures de contrôle à 256 et 512 requêtes non utilisées pour l'ajustement (écart de 2 à 15 %), puis recalé sur les performances publiées du Spark (section 5).

---

3. Le gain en tokens

Mesuré avec le tokenizer de Gemma 4 sur les 704 questions de test et les réponses réelles des deux modèles :

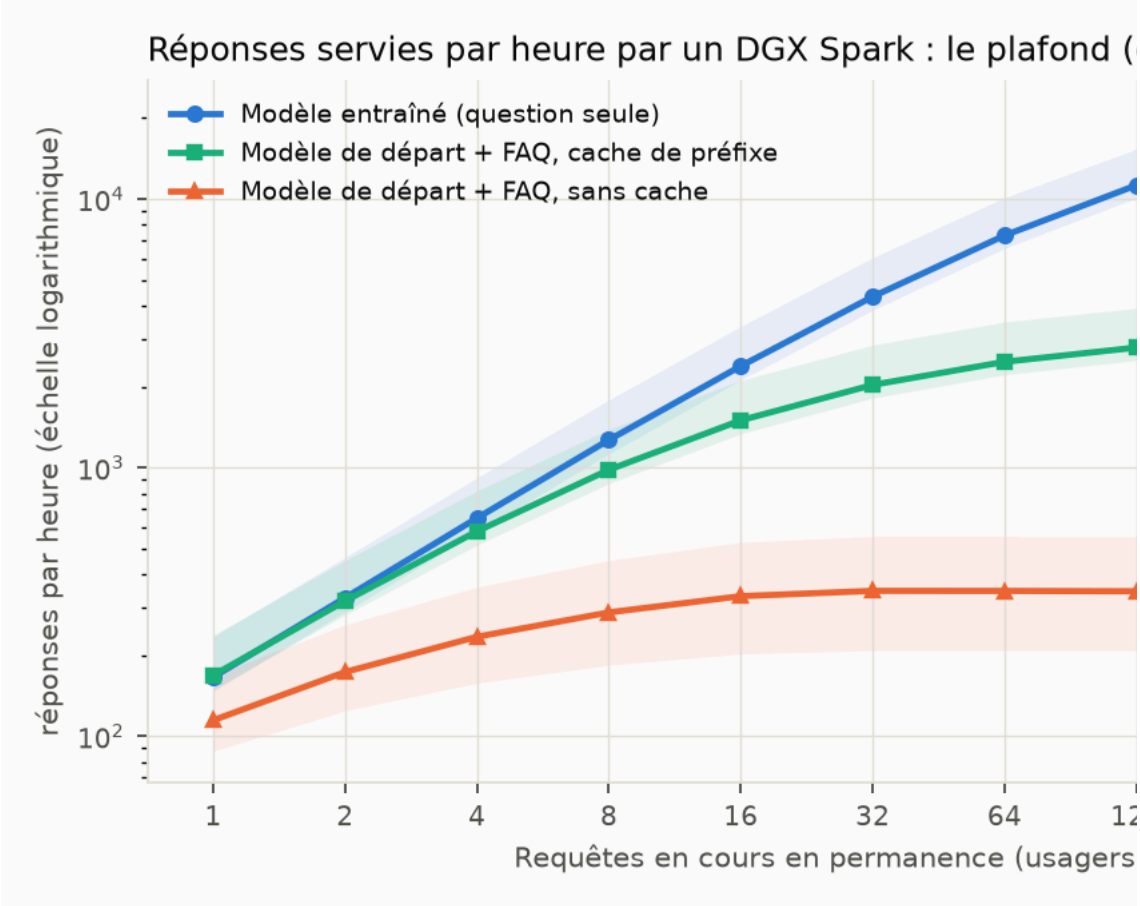
Par question	Modèle entraîné	Modèle de départ + FAQ	Rapport
Consigne système	10 tokens	15 305 tokens (règles + 88 questions/réponses de la FAQ)	
Requête complète (consigne, question, balises)	46 tokens	15 341 tokens	÷ 333
Réponse (moyenne ; médiane)	146 ; 110 tokens	133 ; 106 tokens	
Total traité par échange	192 tokens	15 474 tokens	÷ 81
Tokens à calculer, cache de préfixe actif	46	36 (la FAQ est en cache)	
Mémoire de travail (cache KV) par requête en cours	environ 0,17 Go	environ 1,5 Go	÷ 9

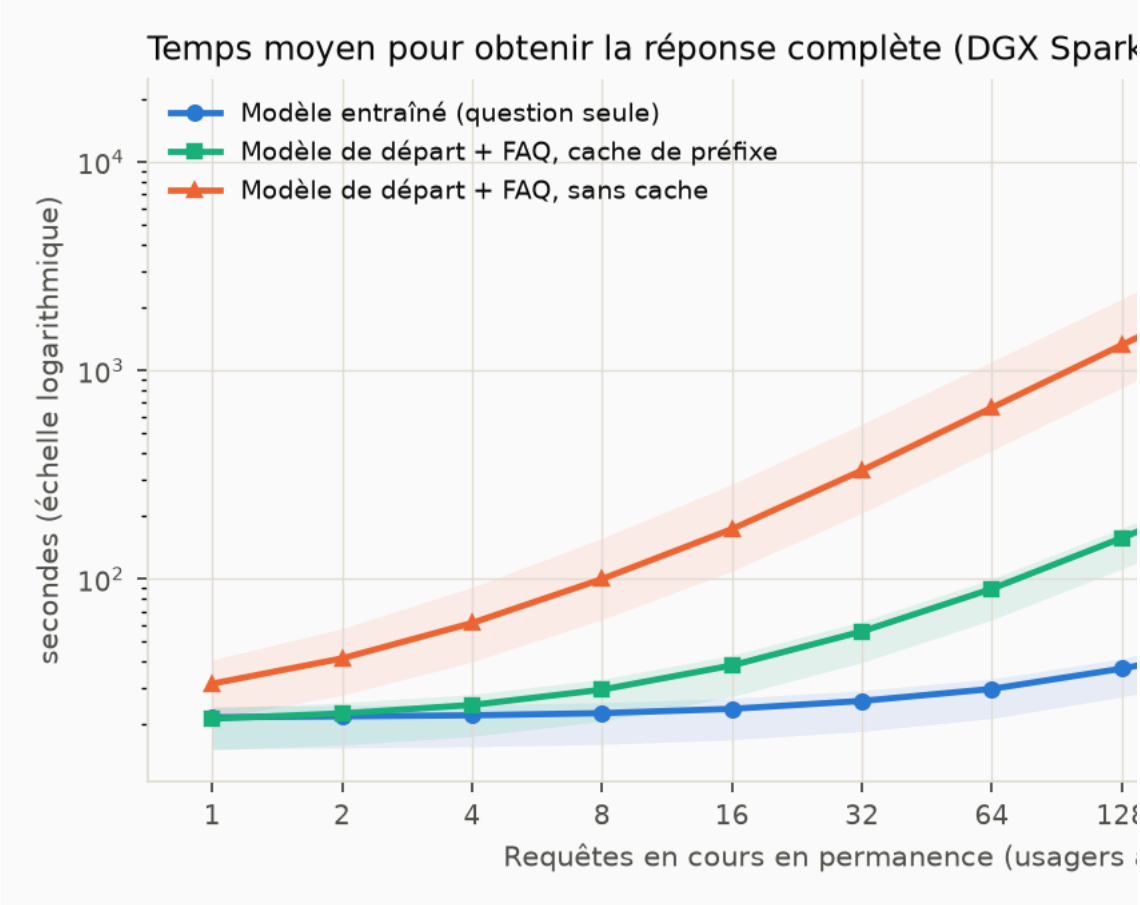
Avec un service facturé au token (API), le rapport ÷ 81 se lit directement en coût par question, sauf si le fournisseur applique un tarif réduit aux tokens en cache. Sur une machine à soi, le cache de préfixe supprime presque tout le calcul de la FAQ, mais pas son coût en mémoire.

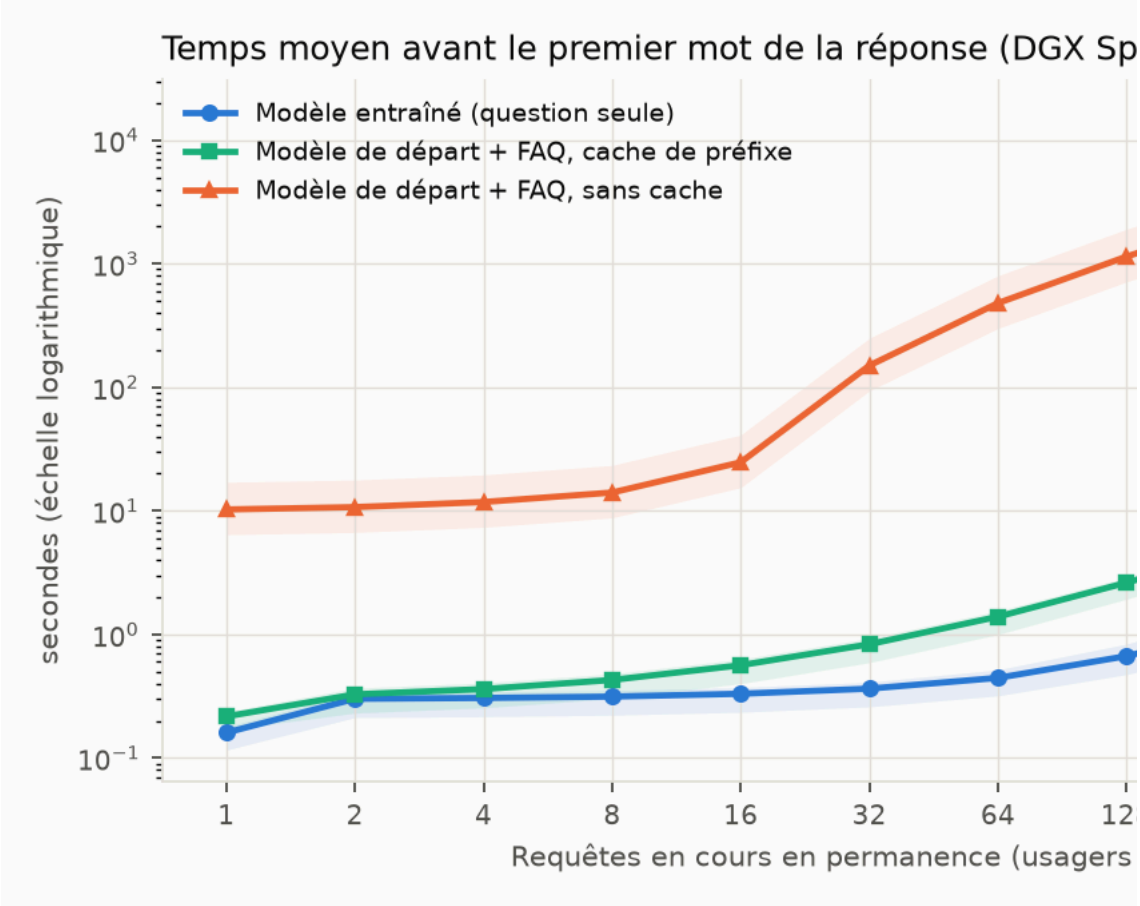
---

4. Capacité et temps de réponse sur un DGX Spark

Régime étudié : N requêtes en cours en permanence (N usagers qui, sitôt leur réponse reçue, sont remplacés par un nouvel usager avec une nouvelle question). Valeurs centrales ; les zones colorées des graphiques couvrent les hypothèses optimiste et pessimiste (section 5.3). Réglages de vLLM par défaut sur le Spark : au plus 256 requêtes traitées en même temps, lots de 2 048 tokens ; au-delà de 256, les requêtes attendent leur tour.




Temps moyen pour obtenir la réponse complète (DGX Spark
Modèle entraîné (question seule)
Modèle de départ + FAQ, cache de préfixe
Modèle de départ + FAQ, sans cache
secondes (échelle logarithmique)
10^4
10^3
10^2
1
2
4
8
16
32
64
128
Requêtes en cours en permanence (usagers


Temps moyen avant le premier mot de la réponse (DGX Sp
Modèle entraîné (question seule)
Modèle de départ + FAQ, cache de préfixe
Modèle de départ + FAQ, sans cache
secondes (échelle logarithmique)
10^4
10^3
10^2
10^1
10^0
10^{-1}
1
2
4
8
16
32
64
12
Requêtes en cours en permanence (usagers

N en cours	Réponses/h : entraîné	Réponses/h : FAQ + cache	Réponses/h : FAQ sans cache	1er mot : entraîné	1er mot : FAQ + cache	1er mot : FAQ sans cache	Réponse : entraîné	Réponse : FAQ + cache	Réponse : FAQ sans cache
1	166	168	114	0,2 s	0,2 s	10 s	22 s	21 s	31 s
2	327	318	173	0,3 s	0,3 s	11 s	22 s	23 s	42 s
4	648	578	234	0,3 s	0,4 s	12 s	22 s	25 s	61 s
8	1 266	980	288	0,3 s	0,4 s	14 s	23 s	29 s	100 s
16	2 385	1 498	332	0,3 s	0,6 s	25 s	24 s	38 s	174 s
32	4 325	2 029	347	0,4 s	0,8 s	150 s	26 s	56 s	332 s
64	7 311	2 474	347	0,4 s	1,4 s	480 s	30 s	90 s	663 s
128	11 251	2 796	346	0,7 s	2,6 s	1 143 s	37 s	157 s	1 324 s
256	15 621	3 007	349	1,2 s	5,2 s	2 465 s	49 s	282 s	2 646 s
512	15 720	2 977	348	55 s	298 s	5 112 s	112 s	600 s	5 294 s
1024	15 417	2 990	348	176 s	924 s	10 403 s	235 s	1 230 s	10 583 s

Lecture :

- **Avant le plafond**, ajouter des usagers augmente le débit presque sans rallonger l'attente pour le modèle entraîné : le Spark lit les poids du modèle (environ 20 Go) une fois pour tous les usagers à chaque pas de calcul. Avec la FAQ, chaque usager ajoute 1,5 Go de cache à relire à chaque pas : le débit monte moins vite et l'attente s'allonge tôt.
- **Au plafond**, chaque usager de plus allonge l'attente de tous, sans gain de débit : le temps de réponse croît alors proportionnellement au nombre de requêtes en cours.
- **Sans cache**, le plafond est atteint dès 32 requêtes en cours : le Spark passe son temps à recalculer la FAQ.
- **Taille de service raisonnable** : pour garder une réponse complète en moins de 40 secondes, un Spark sert environ 11 000 réponses par heure avec le modèle entraîné (128 requêtes en cours), contre environ 1 500 avec la FAQ en cache (16 en cours) ; sans cache, ce délai n'est jamais tenu dès 2 usagers.

Plafonds selon les hypothèses et les réglages

Hypothèse	Entraîné	FAQ + cache	FAQ sans cache	Rapport entraîné / cache	Rapport entraîné / sans cache
optimiste	21 146	4 238	557	5,0	38
central	15 720	3 007	349	5,2	45
pessimiste	13 951	2 702	209	5,2	67
central, vLLM réglé pour le débit	16 798	3 071	339	5,5	49

Réponses par heure au plafond. « vLLM réglé pour le débit » : `--max-num-seqs 1024 --max-num-batched-tokens 8192` au lieu des valeurs par défaut sur GB10 (256 et 2 048). Le gain est faible (7 % pour le modèle entraîné) : le plafond est fixé par la machine, pas par le réglage. Pour le modèle entraîné, la mémoire limite aussi le nombre de requêtes à environ 450 à 500 (80 Go de cache disponibles).

5. Méthode

5.1 Mesures réelles sur H100

Le modèle de départ (Gemma 4 31B, bf16) servi par vLLM 0.34 sur 2 H100 (parallélisme de tenseur 2, au plus 128 requêtes traitées en même temps), interrogé avec les vraies questions de test. Pour que seul le contexte diffère, chaque réponse est forcée à une longueur tirée de la distribution réelle des réponses du modèle correspondant (`ignore_eos`). La vitesse d'un modèle entraîné est celle du modèle de départ pour les mêmes tokens : l'adaptateur LoRA est fusionné, l'architecture est identique. Pour le cas « sans cache », un identifiant aléatoire placé au tout début de la consigne empêche tout partage entre requêtes. Pour le cas « avec cache », une requête préalable a chargé la FAQ (serveur déjà en service). N = 1 à 512 requêtes envoyées en même temps (« sans cache » : jusqu'à 128) ; 28 mesures, 15 minutes au total.

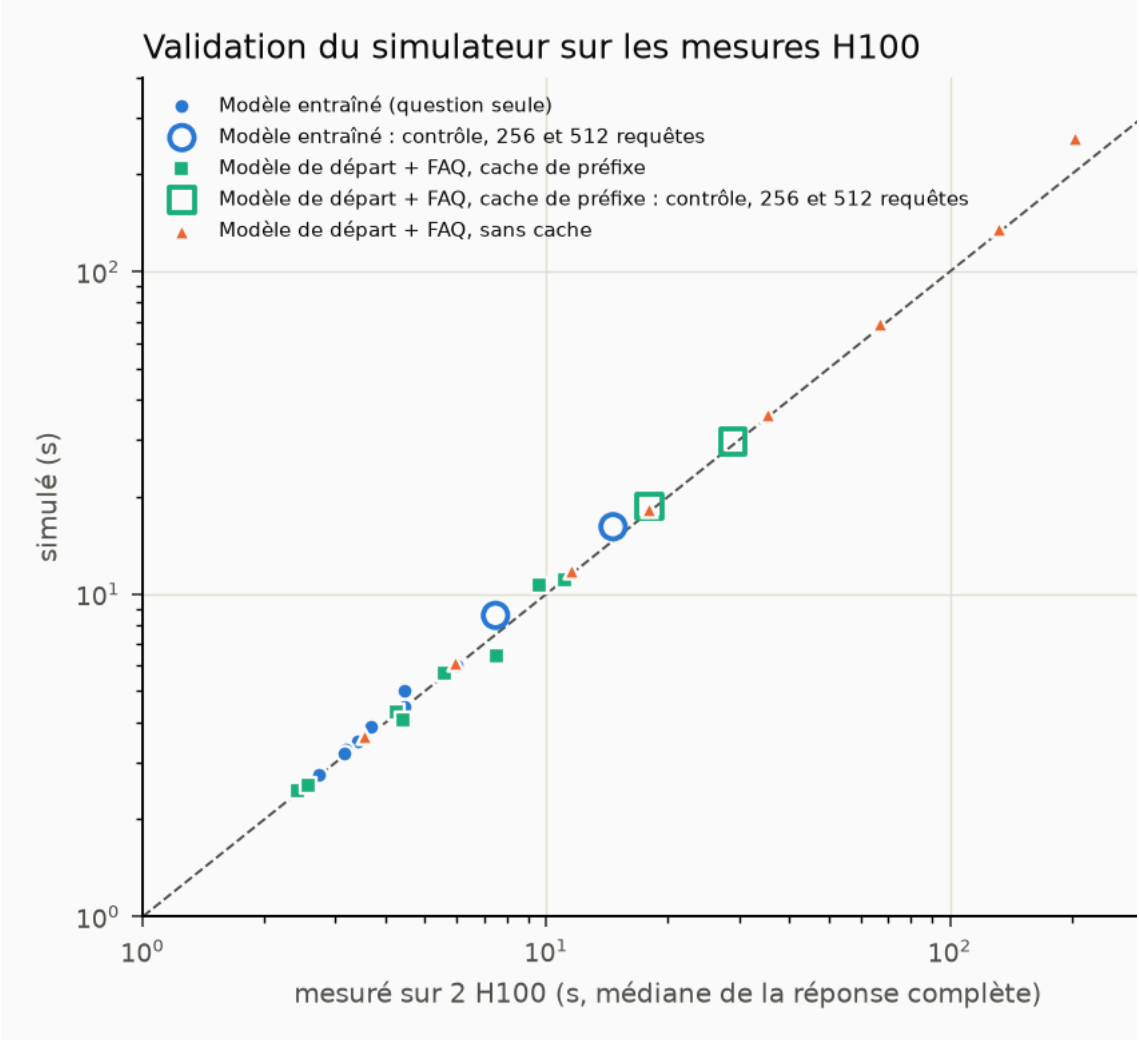
Valeurs mesurées sur H100 à un seul usager : 2,07 s pour lire la FAQ (environ 7 400 tokens par seconde) ; 21,5 ms par token généré sans FAQ, 22,0 ms avec la FAQ en contexte.

5.2 Le simulateur

Le serveur vLLM est simulé pas à pas : à chaque pas, chaque usager en cours de réponse produit un token et le reste du budget de tokens sert à lire des requêtes en attente (« chunked prefill »). La durée d'un pas est le

maximum entre le temps de calcul (opérations des tokens traités et de l'attention, selon l'architecture de Gemma 4 31B : 50 couches à fenêtre glissante de 1 024 tokens et 10 couches à attention globale) et le temps de lecture mémoire (poids plus caches de tous les usagers en cours), plus un coût fixe et un coût processeur par usager en cours (ordonnancement, envoi des mots au fil de l'eau). Le simulateur tient compte de la mise en file par le serveur web (découpage en tokens des requêtes, une par une), de la limite du nombre de requêtes traitées en même temps et de la mémoire disponible pour les caches.

Sur H100, les paramètres sont ajustés sur les mesures à un usager (puissance de calcul effective 492 TFLOPS pour les deux cartes, bande passante effective 3 040 Go/s, 0,9 ms par pas), puis sur les mesures jusqu'à 128 requêtes pour la mise en file (3 µs par token de requête, 10 ms par requête), l'efficacité de lecture des caches (facteur 0,75) et le coût par usager en cours (0,2 ms par pas). Résultat sur le temps de réponse complète : écart médian de 2,7 % sur les 24 mesures ajustées (maximum 28 %, sur le cas « sans cache » à 128 requêtes, où le simulateur est pessimiste pour la solution FAQ) et de 2 à 15 % sur les 4 mesures de contrôle à 256 et 512 requêtes, qui n'ont pas servi à l'ajustement. À 512 requêtes, le rapport mesuré entre la FAQ en cache et le modèle entraîné est de 2,0 sur H100 ; le simulateur donne 1,8 : il ne favorise pas le modèle entraîné.



5.3 Transposition au DGX Spark

Sur le Spark, le modèle est servi en NVFP4 (environ 19,6 Go de poids), le format 4 bits que vLLM exploite sur les puces Blackwell. Les paramètres viennent des mesures publiées :

Paramètre	Central	Optimiste	Pessimiste	Source
Bande passante mémoire effective	134 Go/s	190 Go/s	120 Go/s	Gemma 4 31B NVFP4 sur Spark : 6,85 tokens/s à un usager (ai-muninn.com), soit 19,6 Go × 6,85 ; optimiste : Llama 3.1 70B FP8, 2,7 tokens/s (LMSYS)
Puissance de calcul effective	100 TFLOPS	160 TFLOPS	60 TFLOPS	Llama 3.1 70B FP8, 803 tokens/s en lecture de requête, soit environ 113 TFLOPS sur des requêtes courtes (LMSYS) ; notre requête de 15 000 tokens pèse plus en attention
Mémoire disponible pour les caches	80 Go	80 Go	80 Go	128 Go unifiés, moins le système, les poids et les activations
Requêtes traitées en même temps, budget par pas	256, 2 048 tokens	idem	idem	valeurs par défaut de vLLM sur GB10 (`engine/arg_utils.py`)
Mise en file, efficacité de lecture des caches, coût par usager	comme sur H100			

Contrôle avec des requêtes courtes : le simulateur donne 6,6 tokens par seconde à un usager et 91 au total à 16 usagers ; les mesures publiées pour Gemma 4 31B NVFP4 sont de 6,85 et 105. Le simulateur est donc légèrement pessimiste pour le modèle entraîné.

6. Limites

- **Le DGX Spark n'a pas été mesuré directement** : les temps Spark sont des estimations fondées sur nos mesures H100 et sur des performances publiées par des tiers, obtenues avec d'autres versions de logiciels. Le coût processeur par usager a été mesuré sur les processeurs x86 de la machine H100 ; celui du Spark (processeur ARM) peut différer.
- **La qualité de la version NVFP4 n'a pas été mesurée** ; la version 5 bits (GGUF, llama.cpp, RTX 5090) l'a été (tableau de la section 1). Sur le Spark, cette même version 5 bits peut tourner avec llama.cpp, à une vitesse de génération voisine pour un usager (21,8 Go de poids à lire) ; le traitement simultané de nombreux usagers y est moins optimisé que dans vLLM et n'a pas été simulé.
- Régime le plus chargé : chaque usager servi est aussitôt remplacé par un autre. En usage réel, les questions arrivent au hasard ; le plafond de débit reste le même.
- Une seule question par usager. En conversation, le contexte s'allonge à chaque tour et l'écart se creuse encore pour la solution « FAQ en contexte ».
- La génération spéculative (EAGLE, MTP), qui peut accélérer la génération jusqu'à 2 fois, n'est pas prise en compte ; elle profiterait aux deux solutions.
- Le modèle entraîné ne connaît que la FAQ à la date de sa collecte : à chaque mise à jour de la FAQ, il faut le réentraîner (environ 2 h 30 sur 4 H100 avec la méthode de l'expérience 01 : questions, réponses du professeur, entraînement, quantification), alors que la solution « FAQ en contexte » prend la nouvelle version immédiatement.

7. Reproductibilité

Élément	Fichier (dépôt de l'expérience 01)
Mesures sur H100	`scripts/exp01_bench_faq.py` , résultats `results/exp01_bench_h100.jsonl` , journal `logs/exp01_bench_h100.log`
Simulateur, ajustement et validation H100	`scripts/exp01_sim_serving.py --validate` , `results/exp01_sim_h100.json`
Capacité et temps en régime permanent sur DGX Spark (4 jeux d'hypothèses, N = 1 à 1 024)	`scripts/exp01_sim_serving.py --closed` , `results/exp01_sim_spark_closed.json` , `logs/exp01_sim_closed.log`
Rafales de N questions simultanées (version 1 du rapport)	`scripts/exp01_sim_serving.py --spark` , `results/exp01_sim_spark.json`
Graphiques et tableaux	`scripts/exp01_spark_report.py` , `report/spark/`
Comptage des tokens	tokenizer `google/gemma-4-31B-it` , consigne de l'expérience 01 (`scripts/common.py` , `teacher_system()`)